

System Architect



Prepared by: Ahmed Hassan Ibrahim

LinkedIn Link: <https://www.linkedin.com/in/ahmedhassanese/>



Table of Contents

Data Structure in Embedded Systems:	3
LIFO Buffer:.....	3
Circular Buffer & FIFOS:	5
Linked List:	7
State Machines in C:	15
Moore Vs Mealy State Machines:	15
Ultrasonic Obstacle-avoiding Robot:	16
Modules level:	16
Logical design:	17
SW Logical verification:.....	18
C Implementation:	18
Concepts and Phenomena:	21
System Architecting/Design Sequence:	22
System Design Example: A Pressure Detection System	23
Case Study:	23
SDLC & STLC:	23
Requirements:	24
Space Exploration/ partitioning:	24
Hardware /Software Partitioning:	24
Design Space Exploration:.....	24
System Exploration:	25
System Analysis:	25
Use Case Diagram:	25
Activity Diagram:.....	26
Sequence Diagram:.....	26
System Design:	27
Misra-C:2012 Rules:	30

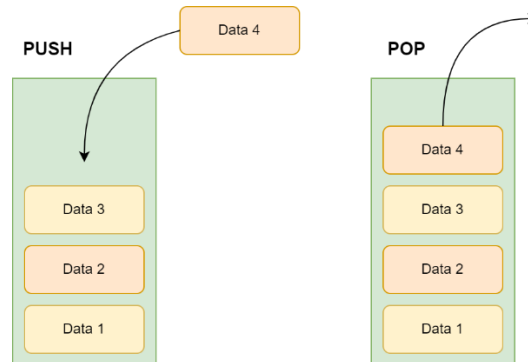


Data Structure in Embedded Systems:

LIFO Buffer:

- **LAST-In-First-Out (LIFO)**
 - Also referred to as a **stack**.
 - Like processor stack, but different.
- Adds and removes data items to and from the **same end**

LIFO Buffer Structure
Base = ?
Head = ?
Lenght = ?



Header File:

```
#ifndef LIFO_H_
#define LIFO_H_

#include "stdio.h"
#include "stdlib.h"

//Type definitions
typedef struct LIFO_Buf
{
    unsigned int length;
    unsigned int count;
    unsigned int* base;
    unsigned int* head;
}LIFO_Buf_t;

typedef enum
{
    LIFO_Passed,
    LIFO_Full,
    LIFO_Empty,
    LIFO_NULL
}LIFO_Status_t;

//APIs
LIFO_Status_t LIFO_Init (LIFO_Buf_t* buffer, unsigned int* arr, unsigned int len);
LIFO_Status_t LIFO_Add_Item (LIFO_Buf_t* buffer, unsigned int item);
LIFO_Status_t LIFO_Get_Item (LIFO_Buf_t* buffer, unsigned int* item);

#endif
```



Implementation code:

```
● ● ●
#include "LIFO.h"

LIFO_Status_t LIFO_Init (LIFO_Buf_t* buffer, unsigned int* arr, unsigned int Len)
{
    if(arr == NULL)
        return LIFO_NULL;

    buffer->base = arr;
    buffer->head = arr;
    buffer->length = Len;
    buffer->count = 0;

    return LIFO_Passed;
}

LIFO_Status_t LIFO_Add_Item (LIFO_Buf_t* buffer, unsigned int item)
{
    if (!buffer->base || !buffer->head)
        return LIFO_NULL;

    if (buffer->count == buffer->length)
        return LIFO_Full;

    *(buffer->head) = item;
    buffer->head++;
    buffer->count++;

    return LIFO_Passed;
}

LIFO_Status_t LIFO_Get_Item (LIFO_Buf_t* buffer, unsigned int* item)
{
    if (!buffer->base || !buffer->head)
        return LIFO_NULL;

    if(buffer->count == 0)
        return LIFO_Empty;

    buffer->head--;
    *item = *(buffer->head);
    buffer->count--;

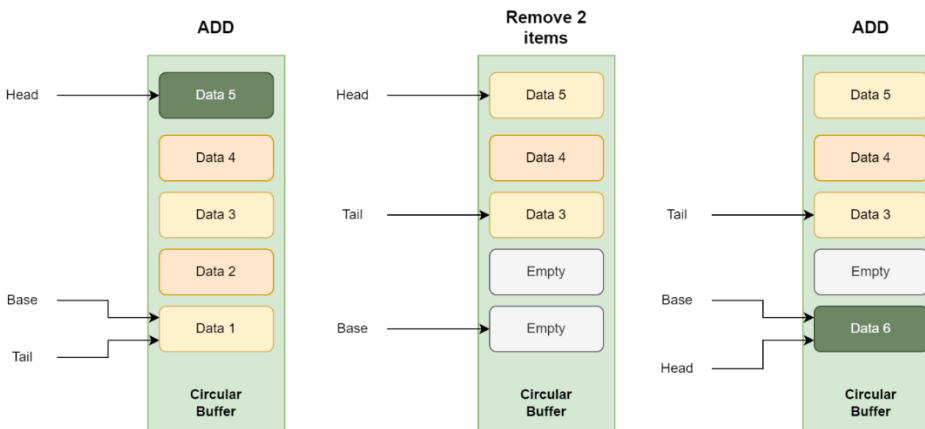
    return LIFO_Passed;
}
```



Circular Buffer & FIFOs:

- We would likely use a circular buffer, or a FIFO-like buffer, to store our **character string to transmit**. And an **interrupt sub-routine to pull that data out**.

Circular FIFO Buffer Structure
Base = ?
Head = ?
Lenght = ?
Tail = ?



Header File:

```

// Header File: FIFO.h

#ifndef FIFO_H_
#define FIFO_H_

#include "stdio.h"
#include "stdint.h"

//<<<<<<< User Configuration >>>>>>>
//Select the element type (uint8_t, uint16_t, uint32_t, ...)
#define element_type uint8_t
//Create buffer
//define width 5
//element_type buff[width];

//FIFO data types
typedef struct FIFO_Buf
{
    uint32_t lenght;
    uint32_t count;
    element_type* tail;
    element_type* head;
    element_type* base;
}FIFO_Buf_t;

typedef enum
{
    FIFO_PASSED,
    FIFO_FULL,
    FIFO_EMPTY,
    FIFO_NULL
}FIFO_Buf_Status;

//FIFO APIs
FIFO_Buf_Status FIFO_Init(FIFO_Buf_t* FIFO, element_type* buffer, uint32_t Len);
FIFO_Buf_Status FIFO_Enqueue(FIFO_Buf_t* FIFO, element_type item);
FIFO_Buf_Status FIFO_Dequeue(FIFO_Buf_t* FIFO, element_type* item);
FIFO_Buf_Status FIFO_Check(FIFO_Buf_t* FIFO);
void FIFO_Print(FIFO_Buf_t* FIFO);

#endif
```




Linked List:

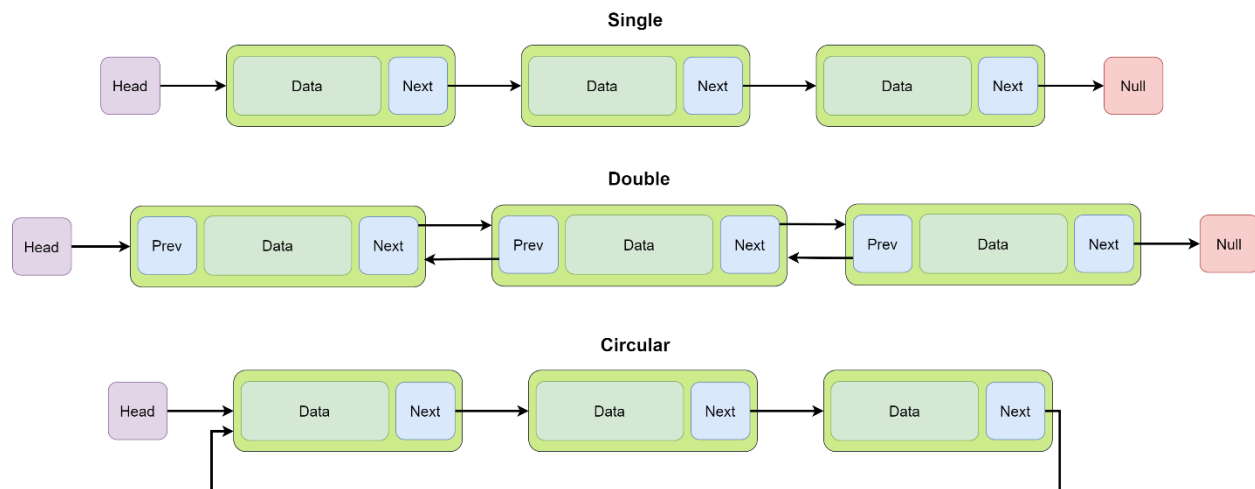
A linked list is a sequence of data structures which are connected via links.

- **Link** – Each Link of a linked list can store data called an element.
- **Next** – Each Link of a linked list contain a link to next link called Next.
- **LinkedList** – A LinkedList contains the connection link to the first Link called First.

Linked List
Next = ?
Head = ?

Types of Linked List:

1. **Simple Linked List** – Item Navigation is forward only.
2. **Doubly Linked List** – Items can be navigated forward and backward way.
3. **Circular Linked List** – Last item contains link of the first element as next and first element has link to last element as prev.





The following are functions implemented for a student database using linked list:

Add Student:

```
void AddStudent()
{
    SStudent_t* p_temp;
    SStudent_t* LastNode;

    // If list is empty
    if (head == NULL)
    {
        // Create new record
        p_temp = (SStudent_t*)malloc(sizeof(SStudent_t));
        // Assign head to new record
        head = p_temp;
    }
    // If not empty
    else
    {
        // Navigate until you reach the last node
        LastNode = head;
        while (LastNode->next_stud != NULL)
            LastNode = LastNode->next_stud;
        // Create new record
        p_temp = (SStudent_t*)malloc(sizeof(SStudent_t));
        // Assign last node to new record
        LastNode->next_stud = p_temp;
    }

    printf("\nEnter student ID: ");
    check_scan_d(p_temp->student.ID);

    printf("\nEnter student full name: ");
    scanf("%[^\\n]*%c", p_temp->student.name);

    printf("\nEnter student height: ");
    check_scan_f(p_temp->student.height);

    p_temp->next_stud = NULL;
}
```




Delete Student:

```
int Delete_Student()
{
    int selected_ID;
    printf("\nEnter student ID to be deleted: ");
    check_scan_d(selected_ID);

    // List is not empty
    if (head)
    {
        SStudent_t* selected_node = head;
        SStudent_t* prev_node = NULL;

        // Loop on all records
        while (selected_node)
        {
            if (selected_node->student.ID == selected_ID)
            {
                // Selected node is not the first node
                if (prev_node)
                    prev_node->next_stud = selected_node->next_stud;
                // Selected node is the first node and the list contain multiple nodes
                else if (selected_node->next_stud)
                    head->next_stud = selected_node->next_stud;

                // Selected node is the first node and the list contain only 1 node
                else
                    head = NULL;

                free(selected_node);
                return 1;
            }

            prev_node = selected_node;
            selected_node = selected_node->next_stud;
        }

        printf("\nProvided ID does not exist in the database.\n");
    }
    else
        printf("\nDatabase is empty.\n");
}
```



View Students:

```
void View_Students()
{
    SStudent_t* Current_Node = head;
    int count = 1;

    if (!head)
        printf("\nDatabase is empty.\n");
    else
    {
        while (Current_Node)
        {
            printf("\nRecord number %d: ", count);
            printf("\n\t ID: %d", Current_Node->student.ID);
            printf("\n\t Name: %s", Current_Node->student.name);
            printf("\n\t Height: %.2f", Current_Node->student.height);
            printf("\n-----\n");
            Current_Node = Current_Node->next_stud;
            count++;
        }
    }
}
```

Delete All:

```
void Delete_All()
{
    SStudent_t* Current_Node = head;

    if (!head)
        printf("\nDatabase is empty.\n");
    else
    {
        while (Current_Node)
        {
            SStudent_t* Temp_Node = Current_Node;
            Current_Node = Current_Node->next_stud;
            free(Temp_Node);
        }
        head = NULL;
    }
}
```



Find Node:

```
int Find_Node()
{
    int index, count = 0;
    SStudent_t* Current_Node = head;
    printf("Enter the nth node you want to search for: ");
    check_scan_d(index);

    while (Current_Node)
    {
        if (count == index)
        {
            printf("\n\t ID: %d", Current_Node->student.ID);
            printf("\n\t Name: %s", Current_Node->student.name);
            printf("\n\t Height: %.2f", Current_Node->student.height);
            return 1;
        }
        count++;
        Current_Node = Current_Node->next_stud;
    }

    printf("\nThe index provided is not valid.\n");
}
```

Find Length:

```
int List_Len(SStudent_t* node)
{
    if (node == NULL)
        return 0;

    return (1 + List_Len(node->next_stud));
}
```



Find Node from End:

```
int Find_Node_From_End()
{
    SStudent_t* Main_Node = head;
    SStudent_t* Reference_Node = head;
    int index, i;

    if (!head)
    {
        printf("\nDatabase is empty.\n");
        return 0;
    }

    printf("Enter the nth node you want to search for from the end: ");
    check_scan_d(index);

    if (index == 0)
    {
        printf("\nThe index provided is not valid.\n");
        return 0;
    }

    //First, move reference pointer to n nodes from head.
    for (i = 0; i < index; i++)
    {
        if (Reference_Node == NULL)
        {
            printf("\nThe index provided is not valid.\n");
            return 0;
        }
        Reference_Node = Reference_Node->next_stud;
    }

    //Now move both pointers one by one until the reference pointer reaches the end.
    //Now the main pointer will point to nth node from the end.
    while (Reference_Node)
    {
        Main_Node = Main_Node->next_stud;
        Reference_Node = Reference_Node->next_stud;
    }

    printf("\n\t ID: %d", Main_Node->student.ID);
    printf("\n\t Name: %s", Main_Node->student.name);
    printf("\n\t Height: %.2f", Main_Node->student.height);
    return 1;
}
```



Find Middle Node:

```
int Find_Middle_Node()
{
    SStudent_t* Fast_Node = head;
    SStudent_t* Slow_Node = head;

    if (!head)
    {
        printf("\nDatabase is empty.\n");
        return 0;
    }

    // Traverse linked list using two pointers.
    // Move one pointer by one and other pointer by two.
    // When the fast pointer reaches end slow pointer will reach middle of the linked list.
    while (Fast_Node)
    {
        Fast_Node = Fast_Node->next_stud;
        if (Fast_Node == NULL)
            break;
        Fast_Node = Fast_Node->next_stud;
        Slow_Node = Slow_Node->next_stud;
    }

    printf("\n\t ID: %d", Slow_Node->student.ID);
    printf("\n\t Name: %s", Slow_Node->student.name);
    printf("\n\t Height: %.2f", Slow_Node->student.height);
    return 1;
}
```



Detect Loop:

```
int Detect_Loop()
{
    SStudent_t* Fast_Node = head;
    SStudent_t* Slow_Node = head;

    if (!head)
    {
        printf("\nDatabase is empty.\n");
        return 0;
    }

    //Move one pointer(slow_p) by one and another pointer(fast_p) by two.
    //If these pointers meet at the same node then there is a loop. If pointers do not meet then linked list doesn't have a loop
    while (Fast_Node)
    {
        Fast_Node = Fast_Node->next_stud;
        if (Fast_Node == NULL)
            break;
        Fast_Node = Fast_Node->next_stud;
        Slow_Node = Slow_Node->next_stud;
        if (Fast_Node == Slow_Node)
        {
            printf("\nLoop is detected\n");
            return 0;
        }
    }
    printf("\nNo loop is detected.\n");
    return 1;
}
```

Reverse List:

```
int Reverse_List()
{
    SStudent_t* Current_Node = head;
    SStudent_t* Prev_Node = NULL;
    SStudent_t* Next_Node = NULL;

    if (!head)
    {
        printf("\nDatabase is empty.\n");
        return 0;
    }

    while (Current_Node)
    {
        Next_Node = Current_Node->next_stud;
        Current_Node->next_stud = Prev_Node;
        Prev_Node = Current_Node;
        Current_Node = Next_Node;
    }

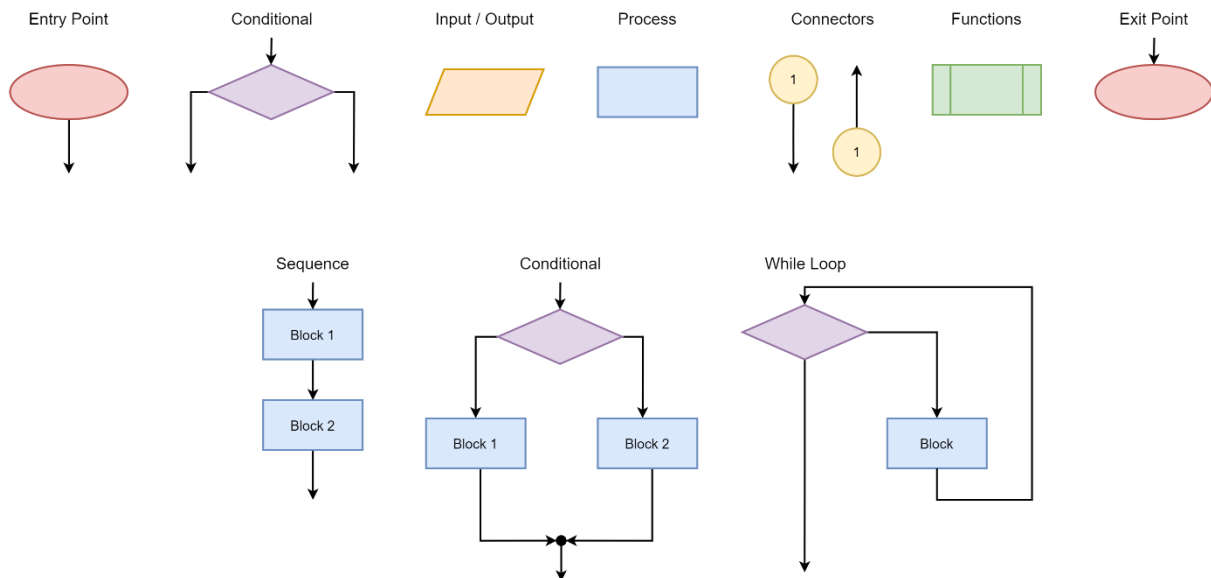
    head = Prev_Node;

    return 1;
}
```



State Machines in C:

Flowchart symbols:

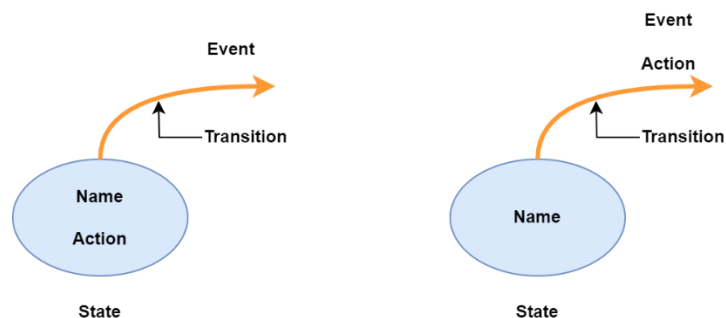


Moore Vs Mealy State Machines:

State machines are perhaps the most effective method for **capturing Time-Ordered Behavior** for embedded system. A state machine has several components:

- Set of **inputs** and **outputs**
- Set of states with **actions**
- Set of **transitions** with conditions
- **Initial state**

Moore	Mealy
Output depends only on the current state.	Output depends on current state and current inputs
<code>outputs = F(currentState);</code>	<code>outputs = F(inputs, currentState);</code>

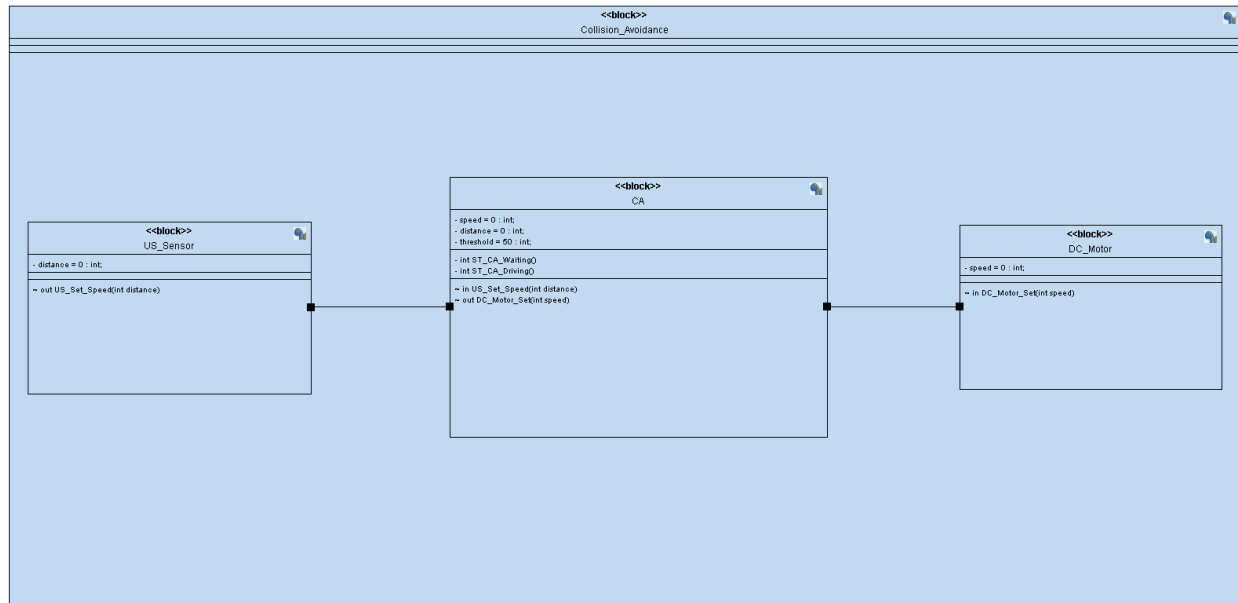




Ultrasonic Obstacle-avoiding Robot:

We will now design a **state diagram** and a **flowchart** for a mini car equipped with an **ultrasonic sensor** and a **DC motor** to control the wheel speed. When the distance drops below a certain threshold, the car will stop; otherwise, it will accelerate up to a specified speed. The design will be created using **TTool**.

Modules level:



1. Block: US_Sensor

- Purpose:** Measures the distance between the car and an obstacle using an ultrasonic sensor.
- Variables:**
 - Distance – stores the measured distance.
- Signals:**
 - out US_Set(int distance) – sends the measured distance to other modules.

2. Block: CA (Collision Avoidance)

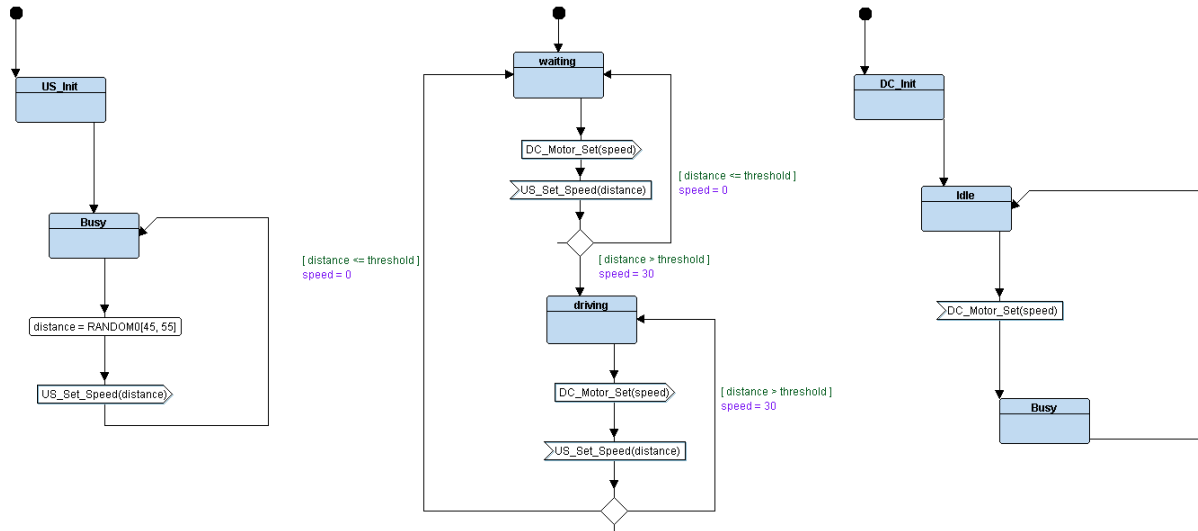
- Purpose:** Implements the decision-making logic to avoid collisions by controlling the car's movement based on distance readings.
- Variables:**
 - Distance – the latest distance value from the ultrasonic sensor.
 - Speed – the car's current speed.
 - Threshold – the minimum safe distance before stopping.
- Signals:**
 - in US_Set(int distance) – receives the measured distance from the ultrasonic sensor.
 - out DC_Motor_Set(int speed) – sends speed commands to the DC motor.



3. Block: DC_Motor

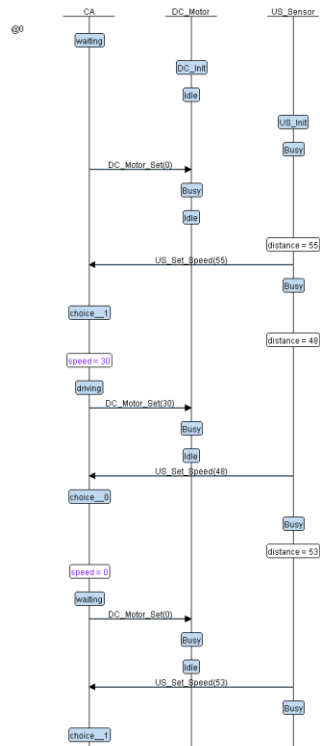
- a. Purpose: Controls the speed of the car's wheels.
- b. Variables:
 - i. Speed – current motor speed setting.
- c. Signals:
 - i. in DC_Motor_Set(int speed) – receives speed commands from the CA module.

Logical design:





SW Logical verification:



C Implementation:

```
C CA.c
h CA.h
C DC.c
h DC.h
C main.c
h state.h
C US.c
h US.h
```



State.h

```
#ifndef STATE_H_
#define STATE_H_

#include "stdio.h"
#include "stdint.h"
#include "stdlib.h"

// Automatic STATE Function Generated
#define STATE_define(_stateFUN_) void ST_##_stateFUN_()
#define STATE(_stateFUN_) ST_##_stateFUN_

// States Connections
void US_Set_Distance(int d);
void DC_Motor_Set(int s);

#endif
```

US.h

```
#ifndef US_H_
#define US_H_

#include "state.h"

// Define states
enum
{
    US_BUSY
};

extern int US_State_ID;
extern void (*US_State)();

// Declare states Functions CA
STATE_define(US_busy);
void US_Init();
int US_Get_Distance_Random(int l, int r, int count);

#endif
```

US.c

```
#include "US.h"

// Variables
static uint32_t US_Distance = 0;
int US_State_ID;
void (*US_State)();

void US_Init()
{
    // Init US Driver
    printf("\nUS_init");
}

STATE_define(US_busy)
{
    // State Name
    US_State_ID = US_BUSY;

    // State Action
    US_Distance = US_Get_Distance_Random(45, 55, 1);

    printf("\nUS_BUSY State: Distance = %d", US_Distance);
    US_Set_Distance(US_Distance);
    US_State = STATE(US_busy);
}

// Function to generate random number in range l and r
int US_Get_Distance_Random(int l, int r, int count)
{
    int i, rand_num;

    for (i = 0; i < count; i++)
        rand_num = (rand() % (r - l + 1)) + l;

    return rand_num;
}
```

DC.h

```
#ifndef DC_H_
#define DC_H_

#include "state.h"

// Define states
enum
{
    DC_IDLE,
    DC_BUSY
};

extern int DC_State_ID;
extern void (*DC_State)();

// Declare states Functions CA
STATE_define(DC_idle);
STATE_define(DC_busy);
void DC_Init();

#endif
```



DC.c

```
● ● ●
#include "DC.h"

// Variables
static uint32_t DC_Speed = 0;

int DC_State_ID;
void (*DC_State)();

void DC_Init()
{
    // Init DC Driver
    printf("\nDC_init");
}

void DC_Motor_Set(int s)
{
    DC_Speed = s;
    DC_State = STATE(DC_busy);
    printf("\nCA----- speed = %d -----> DC", DC_Speed);
}

STATE_define(DC_idle)
{
    // State Name
    DC_State_ID = DC_IDLE;

    // State Action
    // Call PWM to make speed = DC_speed

    printf("\nDC_Idle State: Speed = %d", DC_Speed);
}

STATE_define(DC_busy)
{
    // State Name
    DC_State_ID = DC_BUSY;

    // State Action
    // Call PWM to make speed = DC_speed

    printf("\nDC_Busy State: Speed = %d", DC_Speed);
    DC_State = STATE(DC_idle);
}
```

CA.h

```
● ● ●
#ifndef CA_H_
#define CA_H_

#include "state.h"

// Define states
enum
{
    WAITING,
    DRIVING
};
extern int CA_State_ID;
extern void (*CA_State)();

// Declare states Functions CA
STATE_define(CA_Waiting);
STATE_define(CA_Driving);

#endif
```



CA.c

```
#include "CA.h"

// Variables
static uint32_t CA_Speed = 0;
static uint32_t CA_Distance = 0;
static uint32_t CA_Threshold = 50;
int CA_State_ID;
void (*CA_State)();

void US_Set_Distance(int d)
{
    // Event Check
    CA_Distance = d;
    (CA_Distance <= CA_Threshold) ? (CA_State = STATE(CA_Waiting)) : (CA_State = STATE(CA_Driving));
    printf("\nUS----- distance = %d -----> CA", CA_Distance);
}

STATE_define(CA_Waiting)
{
    // State Name
    CA_State_ID = WAITING;

    // State Action
    CA_Speed = 0;
    DC_Motor_Set(CA_Speed);

    printf("\nCA_Waiting State: Distance = %d    Speed = %d", CA_Distance, CA_Speed);
}

STATE_define(CA_Driving)
{
    // State Name
    CA_State_ID = DRIVING;

    // State Action
    CA_Speed = 30;
    DC_Motor_Set(CA_Speed);

    printf("\nCA_Driving State: Distance = %d    Speed = %d", CA_Distance, CA_Speed);
}
```

main.c

```
#include "CA.h"
#include "DC.h"
#include "US.h"

void setup()
{
    // Init all the drivers
    // Init all IRQ ...
    // Init HAL US_Driver & DC_Driver
    // Init BLOCK
    // Set States Pointers for each Block
    US_Init();
    DC_Init();
    CA_State = STATE(CA_Waiting);
    US_State = STATE(US_busy);
    DC_State = STATE(DC_idle);
}

int main()
{
    setup();

    int i;

    while (1)
    {
        // Call state for each block
        US_State();
        CA_State();
        DC_State();
        for (i = 0; i < 5000; i++);
    }

    return 0;
}
```

Concepts and Phenomena:

➤ Abstraction Vs. Modeling:

- **Abstraction:** Classification of phenomena into concepts
- **Modeling:** Development of abstractions to answer specific questions about a set of phenomena while ignoring irrelevant details.

➤ Verification Vs. Validation:

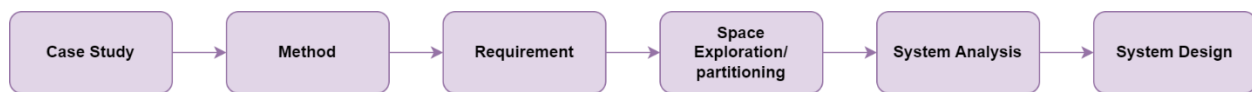
- **Verification** – "Are we building the product right?"
 - Ensures the product is being built according to specifications, design, and requirements.
 - Focuses on correctness of the process and intermediate work products.
 - Examples: code reviews, inspections, unit testing.
- **Validation** – "Are we building the right product?"
 - Ensures the final product meets user needs and intended use in the real environment.
 - Focus on the actual functionality and usefulness to the end user.
 - Examples: system testing, acceptance testing, field trials.



➤ UML Vs. SysML:

- **UML:**
 - A general-purpose modeling language mainly for software engineering.
 - Focuses on modeling object-oriented software systems (classes, components, interactions, states).
- **SysML:**
 - A domain-specific language derived from UML but extended for systems engineering.
 - Models complex multi-disciplinary systems (mechanical, electrical, software, hardware, human, etc.).

System Architecting/Design Sequence:



1. Case Study:

- A real-world problem or scenario used as a **reference example** to understand and apply system architecting methods.
- Help illustrate challenges, constraints, and possible solutions in system design.

2. Method:

- The approach or framework followed to architect/design the system.
- Examples: top-down design, V-model, model-based systems engineering (MBSE), spiral development.

3. Requirement:

- Defines **what the system must achieve** (functional and non-functional needs).
- Comes from stakeholders, users, standards, and environmental constraints.

4. Space Exploration / Partitioning:

- Exploring the **design space** (different possible solutions/alternatives).
- **Partitioning** means dividing the system into **subsystems or modules** for manageability (e.g., hardware/software, control/sensing).
- Helps in identifying trade-offs and optimizing system performance.

5. System Analysis:

- Evaluating the **feasibility, performance, cost, risks, and trade-offs** of proposed designs.
- Uses modeling, simulation, and analytical methods to ensure requirements can be met.
- Helps narrow down alternatives before final design.



6. System Design:

- The process of **transforming requirements and analysis results** in concrete **architecture and detailed specifications**.
- Defines system components, interfaces, data flow, and interactions.
- Results in a **blueprint** for implementation, integration, and testing.

System Design Example: A Pressure Detection System

Case Study:

A” client” expects you to deliver the software of the following system:

- A pressure controller informs the crew of a cabin with an alarm when the **pressure exceeds 20 bars** in the cabin.
- The alarm duration is **60 seconds**.
- **Keeps track of the measured values**.

Pressure Controller: Assumptions

- The controller set up and shutdown procedures are not modeled.
- The controller maintenance is not modeled.
- The pressure sensor never fails.
- The alarm never fails.
- The controller never faces power cut.

SDLC & STLC:

SDLC is Software Development Life Cycle:

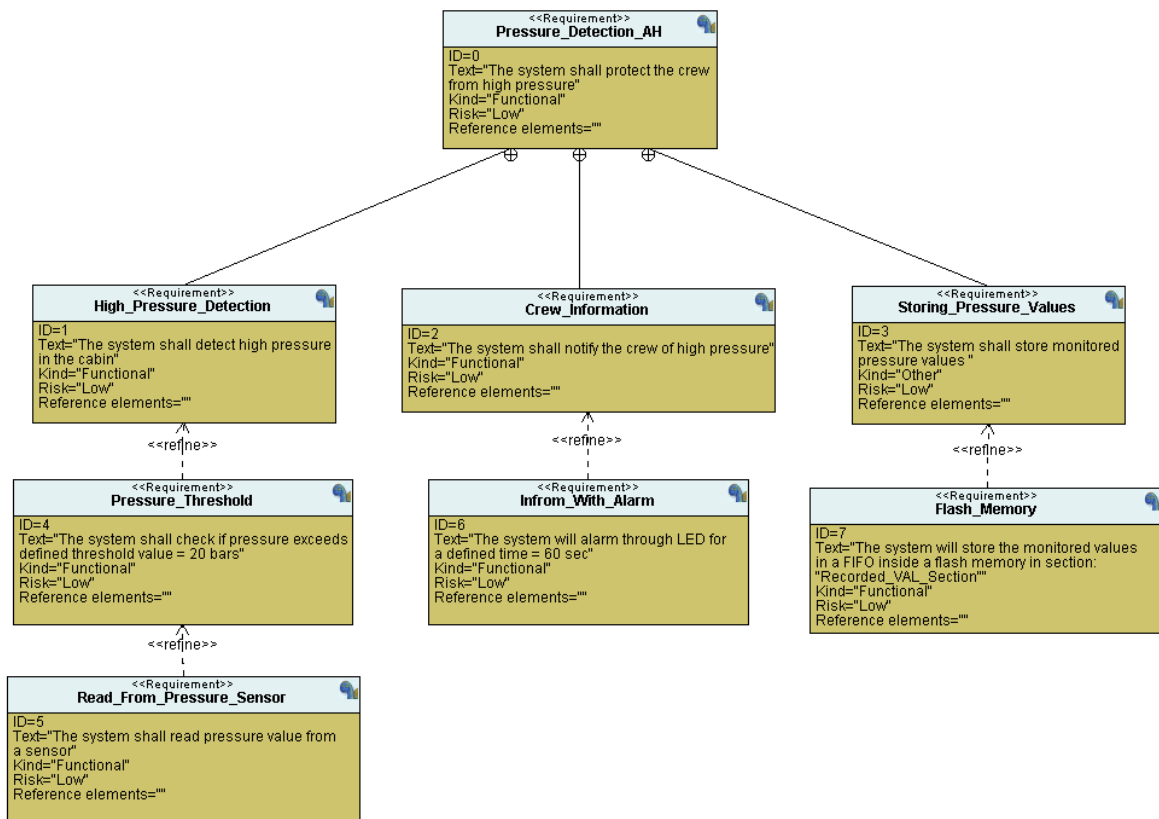
- It is the sequence of activities carried out by Developers to design and develop high-quality software.
- In SDLC, test cases are created.

STLC is Software Testing Life Cycle:

- It consists of a series of activities carried out by Testers methodologically to test your software product.
- In STLC Test cases are executed.



Requirements:



Space Exploration/ partitioning:

Hardware /Software Partitioning:

Definition: A HW/SW partitioning algorithm implements a specification on some sort of multiprocessor architecture

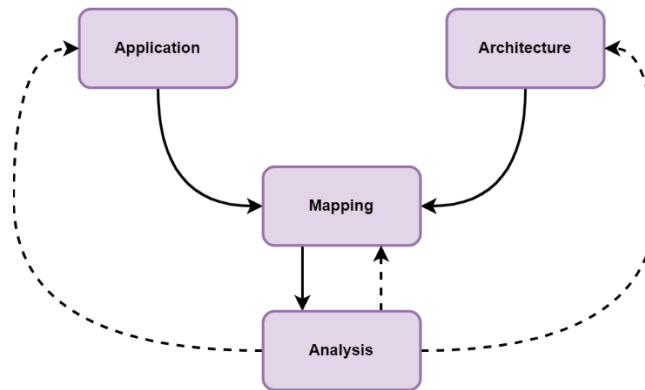
- **Allocation:** Synthesis methods which design the multiprocessor topology along with the Process Elements and SW architecture
- **Scheduling:** The process of assigning Process Element (CPU) time to processes to get executed

Design Space Exploration:

Analyzing various functionally equivalent implementation alternatives and **finding an optimal solution**.



System Exploration:

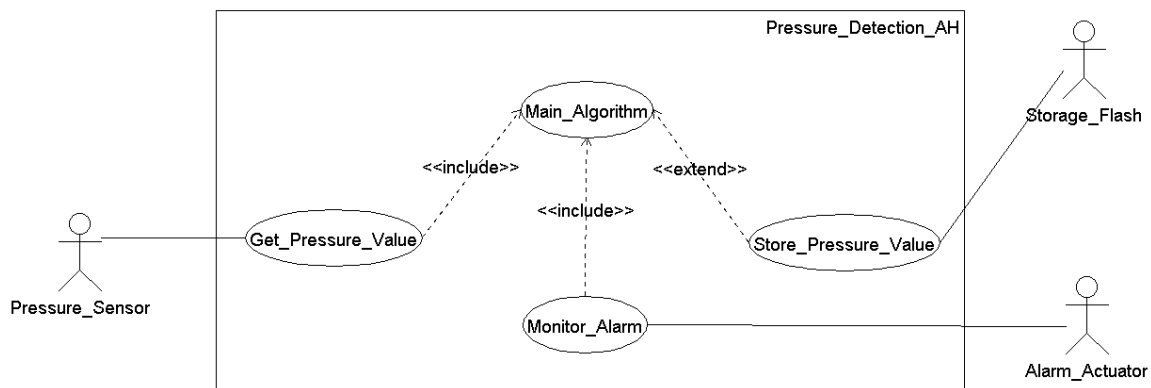


System Analysis:

Analysis method:

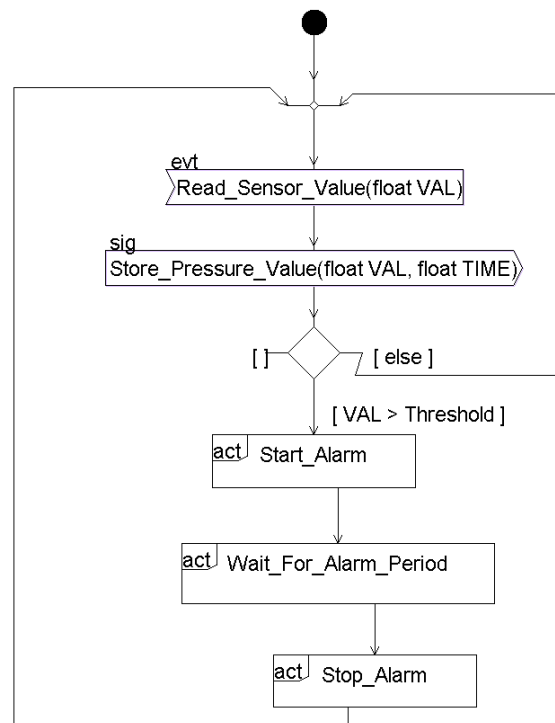
1. System boundary and main functions → [Use Case Diagram](#)
2. Relations between main functions → [Activity Diagram](#)
3. Communications between main system entities and actors → [Sequence Diagram](#)

Use Case Diagram:

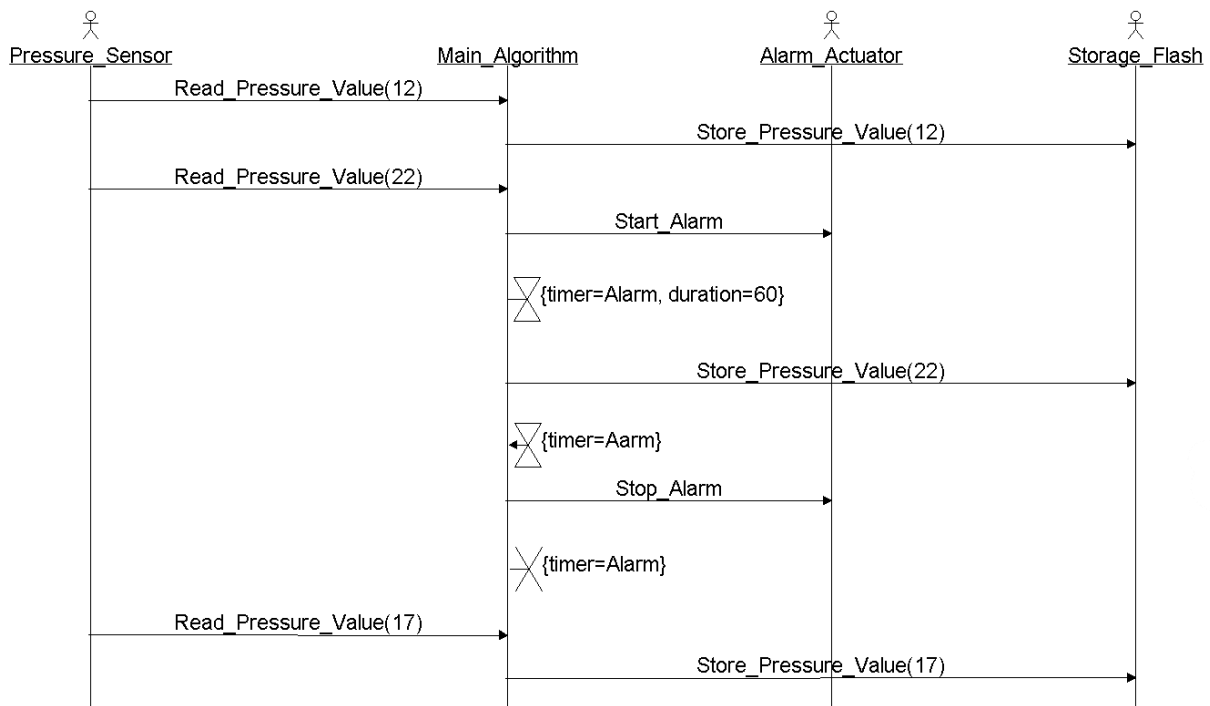




Activity Diagram:



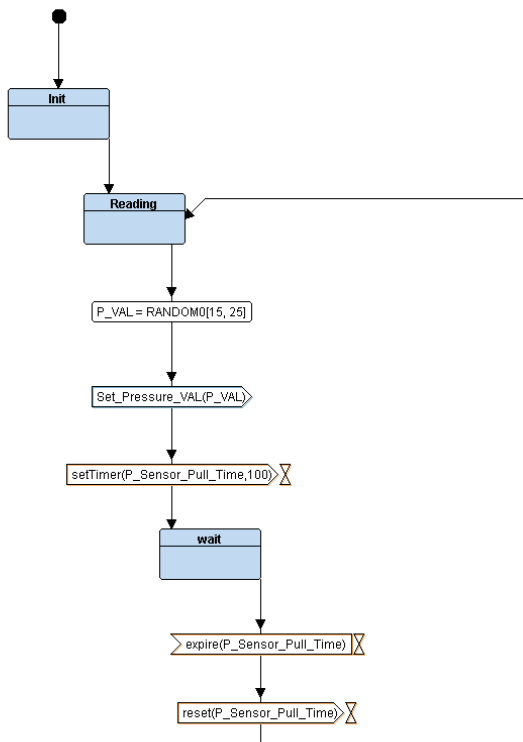
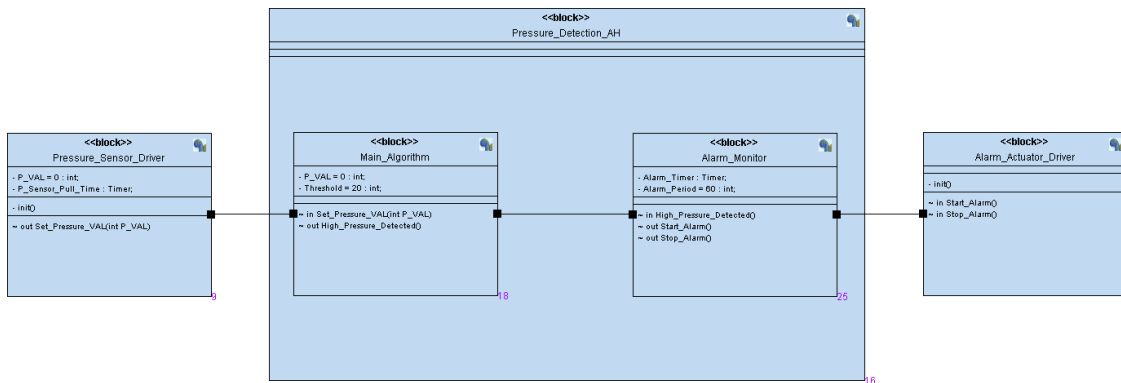
Sequence Diagram:



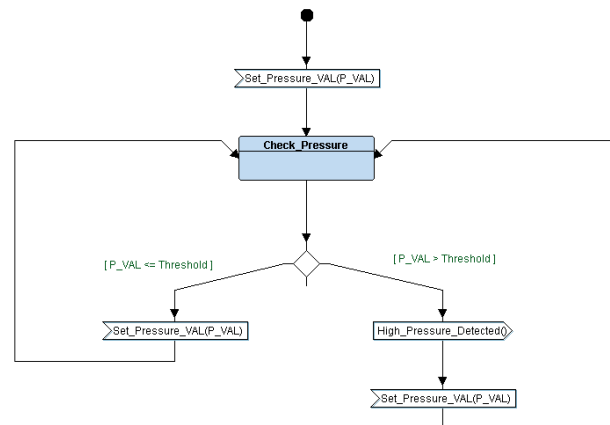


System Design:

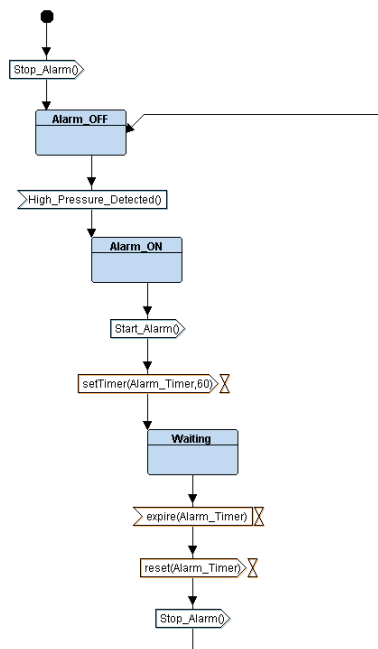
- **Design** → Making what a client wants
- **System architecture** → Block Definition Diagram and Internal Block Diagram
- **Behavior of the system** → State Machine Diagram



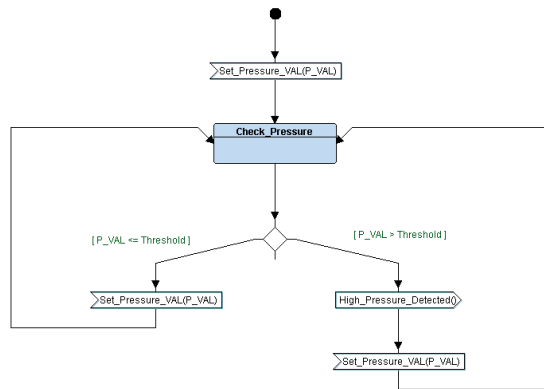
Pressure_Sensor_Driver



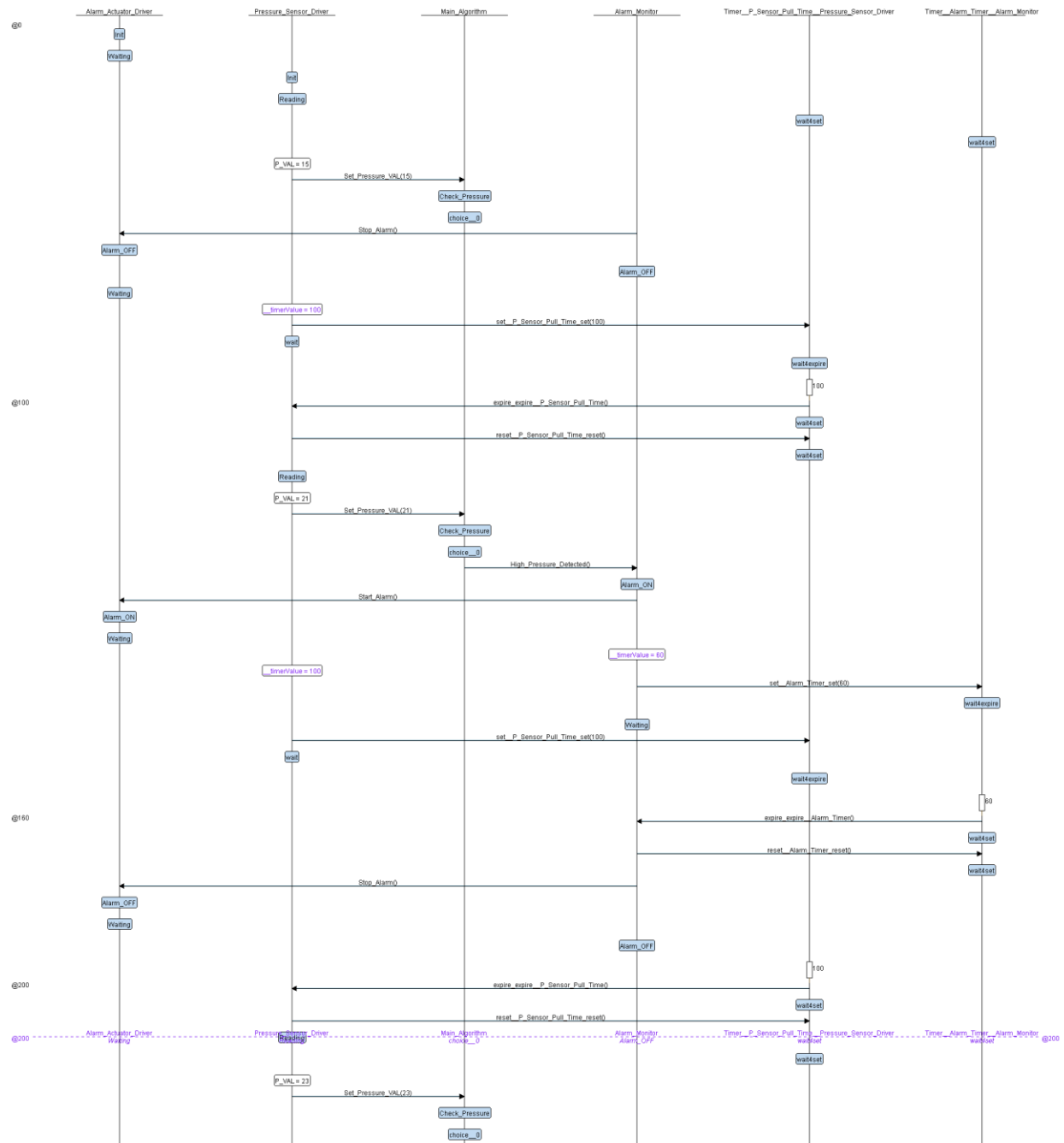
Main_Algorithm



Alarm_Monitor



Alarm_Driver



Trace_Diagram



Misra-C:2012 Rules:

MISRA provides coding standards for developing safety-critical systems

To check if your code follows the Misra-C rules, we will use the **CPP Checker** tool:

- `./cppcheck --dump file.c`
 - This will create `file.c.dump`
- `python addons/addons/misra.py file.c.dump`

Example when testing the following `main.c` code:

```
#include "stdio.h"
#include "stdlib.h"

int main()
{
    int* p = NULL;
    p = (int*) malloc (4);
    free(p);
    return 0;
}
```

```
hp@TyranZilla MINGW64 /d/External Programmes/CPP_Check
$ python addons/addons/misra.py main.c.dump
[main.c:7] (style) misra violation (use --rule-texts=<file> to get proper output) (Undefined) [misra-c2012-21.3]
[main.c:8] (style) misra violation (use --rule-texts=<file> to get proper output) (Undefined) [misra-c2012-21.3]
Checking main.c.dump...
Checking main.c.dump, config ...

MISRA rules violations found:
    Undefined: 2

MISRA rules violated:
    misra-c2012-21.3 (-): 2
```

Now we check `misra-c2012-21.3` violation from documentation:

R.21.2	Required	A reserved identifier or reserved macro name shall not be declared	218 S	Name is used in standard libraries.
			219 S	User name starts with underscore.
R.21.3	Required	The memory allocation and deallocation functions of <code><stdlib.h></code> shall not be used	44 S	Use of banned function, type or variable.
R.21.4	Required	The standard header file <code><setjmp.h></code> shall not be used	43 S	Use of <code>setjmp/longjmp</code> .